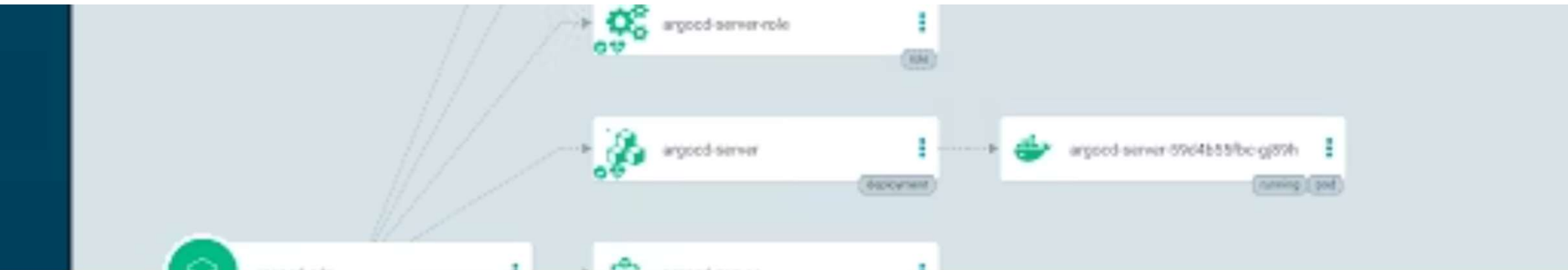


Understanding Argo CD: Kubernetes GitOps Made Simple

Welcome to this comprehensive guide on Argo CD, a Kubernetes-native continuous deployment tool that simplifies GitOps implementation. This presentation will explore how Argo CD works, its key features, installation methods, and best practices for effective adoption.





What Is Argo CD?

Argo CD is a Kubernetes-native continuous deployment (CD) tool that enables GitOps workflows. Unlike external CD tools that only enable push-based deployments, Argo CD can:

- Pull updated code from Git repositories and deploy directly to Kubernetes resources
- Manage both infrastructure configuration and application updates in one system
- Automatically synchronize application state to the current version of declarative configuration
- Detect and remediate configuration drift

Key Features of Argo CD

Deployment Options

- Manual or automatic deployment to Kubernetes clusters
- Automatic synchronization of application state

User Interfaces

- Web user interface for visualization
- Command-line interface (CLI)

Security & Management

- Role-based access control (RBAC)
- Multi-cluster management capabilities
- Single sign-on with multiple providers

Integration

- Support for webhooks (GitLab, GitHub, BitBucket)
- Compatibility with existing tools (YAML, Helm, Kustomize)

These features make Argo CD a powerful tool for implementing GitOps workflows in Kubernetes environments.

Understanding GitOps with Argo CD

GitOps is a software engineering practice that uses a Git repository as the single source of truth for declarative infrastructure and applications.



Git Repository

Contains all declarative configurations



Pull Requests

Changes introduced via PRs



Merge to Main

Configurations merged to main branch



Auto Deployment

Changes automatically deployed

Argo CD handles the latter stages of this process, ensuring that new configurations are correctly deployed to Kubernetes clusters while maintaining a complete audit trail of all changes.

The Argo CD Process

At a high level, the Argo CD process works like this:

1. Developer makes changes to an application, pushing a new version of Kubernetes resource definitions to a Git repo
2. Continuous integration is triggered, resulting in a new container image saved to a registry
3. Developer issues a pull request, changing Kubernetes manifests
4. Pull request is reviewed and changes are merged to the main branch, triggering a webhook
5. Argo CD clones the repo and compares application state with the current state of the Kubernetes cluster
6. Kubernetes uses its controllers to reconcile the changes required to cluster resources
7. Argo CD monitors progress and reports when the application is in sync
8. Argo CD also monitors changes in the Kubernetes cluster and discards them if they don't match Git configuration

How Argo CD Makes GitOps Happen

GitOps Agent

Pulls updated code from Git repositories and deploys directly to Kubernetes resources, managing both infrastructure configuration and application updates in one system

Custom Resource Definitions (CRDs)

Extends the Kubernetes API to define application state declaratively based on Git or Helm repositories

Command Line Interface

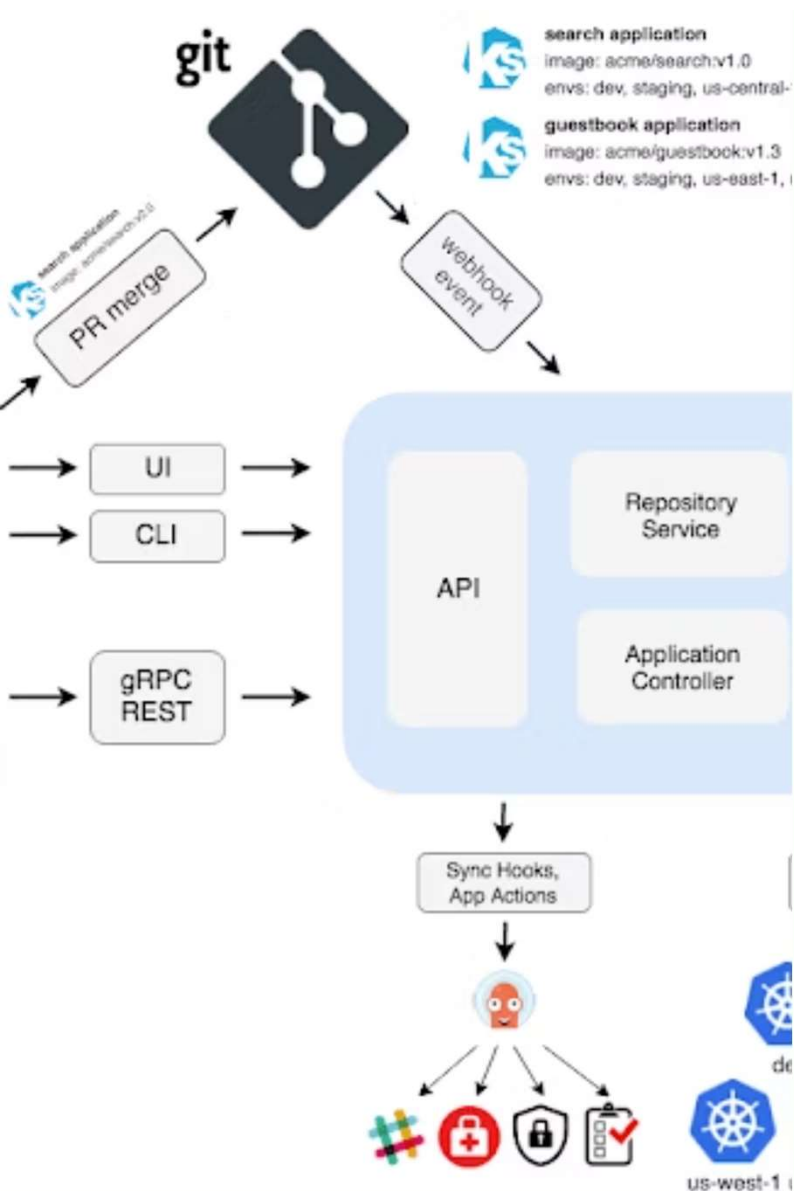
Powerful CLI that creates YAML resource definitions with simple commands

User Interface

Web-based UI for defining applications and visualizing Kubernetes configurations

Multi-tenancy Support

Strong support for multiple teams working on different projects in the same Kubernetes environment



Argo CD Architecture

Argo CD is a Kubernetes controller that continuously monitors applications, comparing their live state to the desired state in Git. It identifies applications with deviations as "OutOfSync" and provides tools to reconcile these differences either manually or automatically.

Core Components of Argo CD

API Server

- Exposes gRPC/REST API for CLI and Web UI
- Manages applications and reports status
- Invokes application operations (sync, rollback)
- Manages cluster and repository credentials
- Handles authentication and authorization
- Enforces RBAC policies
- Processes Git webhook events

Repository Service

- Caches Git repositories locally
- Stores application manifests
- Generates Kubernetes manifests
- Returns manifests based on repository URL, path, revisions, and template settings

Application Controller

- Continuously monitors applications
- Compares target state with current state
- Identifies OutOfSync applications
- Implements corrections when specified
- Invokes user-defined lifecycle hooks



Expert Tips for Argo CD

1 Automate Secret Management

Integrate tools like HashiCorp Vault or AWS Secrets Manager with Argo CD to handle secrets dynamically, ensuring secure management without manual intervention.

2 Integrate with Existing CI Pipelines

Use Argo CD with CI tools like Jenkins or GitHub Actions to create seamless CI/CD pipelines that trigger deployments automatically when changes are pushed.

3 Implement Policy Enforcement

Use Open Policy Agent (OPA) with Gatekeeper to enforce policies on Kubernetes resources, ensuring all deployments adhere to organizational standards.



More Expert Tips

1 Utilize Argo Rollouts for Advanced Deployments

Combine Argo CD with Argo Rollouts to implement advanced deployment strategies such as blue-green, canary, or progressive delivery for more control and reduced risk.

2 Monitor Git Repositories for Security

Regularly scan Git repositories for vulnerabilities and misconfigurations using tools like Snyk or Trivy integrated into your pipeline.

These tips from [Dan Garfield, VP of Open Source at Octopus Deploy](#), can help you maximize the benefits of Argo CD in your organization.



How Argo CD Works with Kubernetes Manifests

Argo CD supports multiple types of Kubernetes manifests:

- Plain YAML or JSON manifests
- Helm charts
- Customize applications
- Ksonnet and Jsonnet applications
- Custom configuration management tools (via plugins)

Argo CD automatically deploys the desired state of an application in a specified target environment. Updates are traceable through Git commits, tags, branches, or specific versions of a manifest.

Argo CD Installation Methods

1

Core Installation

- Best for cluster admins using Argo CD independently
- Fewest components, easy setup
- No UI or API server included
- Not suitable for multi-tenancy
- Doesn't support high availability

```
kubectl create namespace argocd
```

```
kubectl apply -n argocd -f
```

```
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/core-install.yaml
```

2

Multi-Tenant Installation

- Most popular installation type
- Serves multiple Dev teams
- Requires platform team maintenance
- API server allows CLI or Web UI access
- Available in HA and non-HA versions

HA recommended for production use

Additional Installation Methods

Kustomize Installation

Install Argo CD manifests using Kustomize:

```
kind: Kustomizationnamespace: argocdresources:- https://raw.githubusercontent.com/argoproj/argo-cd/master/manifests/ha/install.yaml
```

Organizations should maintain the manifest as a remote resource and use Kustomize patches to apply customizations.

Helm Installation

Argo CD can also be installed via Helm chart. Open source community-maintained Helm charts are available at:

[argo-helm/charts/argo-cd](https://argo-helm.github.io/charts/argo-cd)

This method is ideal for organizations already using Helm for other Kubernetes deployments, providing a consistent installation approach.



Best Practice: Separate Source Code and Configuration



Source Code Repository

Application code and business logic



Configuration Repository

Kubernetes manifests and deployment configs

Benefits of Separation:

- More manageable repositories - modify one without affecting the other
- Cleaner logs for auditing purposes - easier to trace Git history
- Enhanced security - restrict access to source code and Kubernetes manifests
- Support for different versioning and release cycles in microservices
- Prevention of build job loops in automated CI pipelines

Best Practice: Choose the Right Number of Config Repos

Small Companies

- Don't rely heavily on automation
- All employees are trusted
- Recommendation: Use a mono-repo

Mid-sized Companies

- Use some automation
- Multiple teams with different needs
- Recommendation: One repository per team

Large Organizations

- Require greater control
- Rely significantly on automation
- Recommendation: One repository per service

Teams can often manage themselves better with their own repository, deciding who has release access without creating central bottlenecks.

Best Practice: Test Manifests Before Committing

Why Test Before Committing?

- Prevents introduction of issues into pre-production environments
- Reduces failed deployments and troubleshooting time
- Improves overall stability of the GitOps workflow

Testing Approach:

- Run commands locally to test manifests before committing changes
- Validate Helm charts or other templates used to generate manifests
- Ensure syntax and semantic correctness before pushing to the repository



Best Practice: Prevent External Changes to Manifests

The Configuration Drift Problem:

- Ad-hoc commands outside the CI/CD process cause configuration drift
- Drift impacts application deployments - apps may pass in staging but fail in production
- Changes made directly with `kubectl` bypass the GitOps workflow

How Argo CD Helps:

- Uses Git as a single source of truth for all deployments
- Maintains state traceability through Git history
- Detects and marks applications as OutOfSync when direct changes are made
- Offers auto-sync capability to eliminate configuration drift

Important: Ensure all manifest changes are committed to the Git repository to maintain a clean history.

Argo CD in the GitOps Ecosystem

CI Tools

Build artifacts and update manifests

Argo CD

Syncs Git state to clusters



Git Repos

Store manifests, charts, and configs

Container Registry

Host built container images

Kubernetes & Monitoring

Run apps and observe health

Related Argo Projects



Argo Rollouts

Extends Kubernetes with advanced deployment capabilities like canary, blue-green, and progressive delivery strategies.



Argo Workflows

Container-native workflow engine for orchestrating parallel jobs on Kubernetes, useful for CI/CD, data processing, and machine learning.



Argo Events

Event-driven workflow automation framework for Kubernetes that helps you trigger workflows from a variety of events.

These projects complement Argo CD to provide a comprehensive GitOps platform for Kubernetes.

Summary: Why Choose Argo CD for GitOps

Kubernetes-Native

Purpose-built for Kubernetes, leveraging its controller pattern and extending its API

GitOps Automation

Automates the deployment process using Git as the single source of truth

Configuration Drift Prevention

Detects and remediates configuration drift to maintain system stability

Visualization & UI

Provides powerful visualization tools to understand application state

Enterprise Readiness

Supports multi-tenancy, RBAC, and high availability for production use

Argo CD simplifies Kubernetes GitOps implementation, making continuous deployment more reliable, traceable, and manageable.